Modular Arithmetic

Beulah Evanjalin

Contents

1 Introduction

Modular arithmetic, often referred to as *clock arithmetic*, is a foundational tool in mathematics, particularly when working within finite algebraic structures.

Modular arithmetic concerns operations within the ring of integers modulo m, denoted $\mathbb{Z}/m\mathbb{Z}$, where arithmetic is performed under the equivalence relation $a \equiv b \pmod{m}$. This equivalence partitions the integers into congruence classes, facilitating operations where values are periodically identified.

1.1 Applications

Many public-key cryptographic protocols rely intrinsically on modular arithmetic due to its favorable computational properties and algebraic hardness assumptions.

- **RSA Cryptosystem**: Based on Euler's theorem, RSA operates with a modulus $n = pq$, where p and q are large primes. Given e such that $\gcd(e, \phi(n)) = 1$, encryption is given by $c \equiv m^e \pmod{n}$, and decryption via $m \equiv c^d \pmod{n}$, where $d \equiv e^{-1} \pmod{\phi(n)}$.
- **Diffie-Hellman Key Exchange**: Employs modular exponentiation in a finite cyclic group $\mathbb{Z}_p^*$, enabling secure computation of shared secrets over insecure channels by leveraging the discrete logarithm problem.
- **Elliptic Curve Cryptography (ECC)**: Extends modular arithmetic to elliptic curves over finite fields $\mathbb{F}_p$ or $\mathbb{F}_{2^m}$, offering higher cryptographic strength per bit through richer group structures.

Note: In each of these above schemes, the feasibility of operations like $a^b \bmod m$ for very large integers requires efficient exponentiation algorithms, such as binary exponentiation (a.k.a. square-and-multiply).

- **Hash Functions**: Use modular reduction to constrain outputs to a fixed-size range, as seen in hash tables and cryptographic digests.
- **Random Number Generators**: Pseudorandom sequences, such as those from linear congruential generators, are defined recursively as $x_{n+1} \equiv ax_n + c \pmod{m}$.
- **Finite Field Arithmetic**: Key to error-correcting codes like Reed-Solomon, which operate over $\mathbb{F}_q$, often implemented using modular arithmetic over prime fields.
- **Big Integer Libraries**: Efficient handling of arithmetic with large numbers often requires modular reductions to manage intermediate computations and memory usage.

1.2 Role in Number Theory

Within mathematics, modular arithmetic provides both a practical tool and a conceptual framework:

- **Fermat's Last Theorem**: Wiles' proof is based on modular forms and elliptic curves, where congruence relations play a central role.
- **Distribution of Primes**: Properties such as Dirichlet's theorem on arithmetic progressions and the properties of residues mod n shape analytic and algebraic number theory.
- **Solving Diophantine Equations**: Modulo constraints reduce complex integer problems to manageable local conditions, facilitating techniques like Hensel's lemma and the method of descent.

Abstractly, modular systems exemplify rings, fields (when modulus is prime), and groups, which provide concrete cases for studying general algebraic structures.

1.3 Historical Origins

Although modular arithmetic was formalized in the 19^{th} century, its roots extend to antiquity.

- **Early Chinese Mathematics**: *Sunzi Suanjing* (circa 100 BCE) outlines problems equivalent to the modern Chinese Remainder Theorem (CRT). For example, solving:

$$\begin{cases} x \equiv 2 \pmod 3 \\ x \equiv 3 \pmod 5 \\ x \equiv 2 \pmod 7 \end{cases}$$

  yields $x \equiv 23 \pmod{105}$, showcasing congruential reasoning.
- **Gauss (1801)**: In *Disquisitiones Arithmeticae*, Gauss introduced the formal congruence notation $a \equiv b \pmod m$, established properties such as the law of quadratic reciprocity, and laid the foundation for modern number theory.
- **Euler and Fermat**: Fermat's Little Theorem and Euler's generalization via the totient function $\phi(n)$ demonstrated the periodic nature of powers modulo primes and composites, which is fundamental to later cryptographic applications.
- **Computational Era**: In the mid 20^{th} century, there was a need for more efficient algorithms, such as exponentiation by squaring. By the 1970s, cryptographic primitives like RSA and ElGamal leveraged modular arithmetic for secure digital communication.
- **Quantum and Post-Quantum**: Even in the face of quantum algorithms (e.g., Shor's), many post-quantum cryptographic schemes retain modular structures in lattice-based and code-based cryptography, indicating its enduring utility.

2 Basics of Congruences

Definition 2.1. Let $m \geq 1$ be an integer (the **modulus**). We say that two integers a and b are **congruent modulo** m, denoted $a \equiv b \pmod{m}$, if m divides $a - b$, i.e., $m \mid (a - b)$.

Equivalently, there exists an integer k such that $a = b + km$. This relation captures the notion of "remainder" when dividing by m: a and b leave the same remainder upon division by m.

Example 2.1.

- $15 \equiv 3 \pmod{12}$ because $15 - 3 = 12$, and $12 \mid 12$.
- $-1 \equiv 11 \pmod{12}$ because $-1 - 11 = -12$, and $12 \mid -12$.
- $17 \equiv 2 \pmod{5}$ because $17 = 3 \cdot 5 + 2$, so $17 - 2 = 15$, and $5 \mid 15$.
- However, $19 \not\equiv 6 \pmod{11}$ because $19 - 6 = 13$, and $11 \nmid 13$.

2.1 Equivalence Relation

Congruence modulo m defines an equivalence relation on the set of integers $\mathbb{Z}$, partitioning it into disjoint classes.

Theorem 2.1. *Congruence modulo m is an equivalence relation on $\mathbb{Z}$, satisfying:*

- ***Reflexivity**: $a \equiv a \pmod{m}$ for all $a \in \mathbb{Z}$.*
- ***Symmetry**: If $a \equiv b \pmod{m}$, then $b \equiv a \pmod{m}$.*
- ***Transitivity**: If $a \equiv b \pmod{m}$ and $b \equiv c \pmod{m}$, then $a \equiv c \pmod{m}$.*

Proof.

- **Reflexivity**: $a - a = 0$, and $m \mid 0$ (since $0 = m \cdot 0$).
- **Symmetry**: If $m \mid (a - b)$, then $a - b = km$ for some integer k, so $b - a = (-k)m$, hence $m \mid (b - a)$.
- **Transitivity**: If $m \mid (a - b)$ and $m \mid (b - c)$, then $a - b = km$ and $b - c = lm$ for integers k, l. Thus, $a - c = (a - b) + (b - c) = (k + l)m$, so $m \mid (a - c)$.

□

2.2 Residue Classes

This equivalence relation partitions $\mathbb{Z}$ into m distinct **residue classes** (or **congruence classes**).

Definition 2.2. The **residue class** of a modulo m is the set

$$[a]_m = \{b \in \mathbb{Z} : b \equiv a \pmod{m}\} = \{a + km : k \in \mathbb{Z}\}.$$

The set of all residue classes modulo m is denoted $\mathbb{Z}/m\mathbb{Z}$.

There are exactly m distinct residue classes, which can be represented by the remainders $0, 1, \ldots, m-1$ when dividing by m. For any integer a, we can write $a = qm + r$ with $0 \leq r < m$ (by the Division Algorithm), so $[a]_m = [r]_m$.

Example 2.2. For $m = 5$, the residue classes are:

- $[0]_5 = \{\ldots, -10, -5, 0, 5, 10, \ldots\}$
- $[1]_5 = \{\ldots, -9, -4, 1, 6, 11, \ldots\}$
- $[2]_5 = \{\ldots, -8, -3, 2, 7, 12, \ldots\}$
- $[3]_5 = \{\ldots, -7, -2, 3, 8, 13, \ldots\}$
- $[4]_5 = \{\ldots, -6, -1, 4, 9, 14, \ldots\}$

Note that $17 \equiv 2 \pmod{5}$, so $[17]_5 = [2]_5$.

2.3 Arithmetic of Congruences

Congruences behave similarly to equalities under addition, subtraction, and multiplication.

Proposition 2.1. *If $a_1 \equiv a_2 \pmod{m}$ and $b_1 \equiv b_2 \pmod{m}$, then*

$$a_1 + b_1 \equiv a_2 + b_2 \pmod{m}, \quad a_1 - b_1 \equiv a_2 - b_2 \pmod{m}, \quad a_1 b_1 \equiv a_2 b_2 \pmod{m}.$$

Proof. Since $a_1 \equiv a_2 \pmod{m}$, we have $a_1 = a_2 + km$ for some integer k. Similarly, $b_1 = b_2 + lm$ for some l.

- Addition: $a_1 + b_1 = (a_2 + b_2) + (k + l)m$, so $m \mid [(a_1 + b_1) - (a_2 + b_2)]$.
- Subtraction: $a_1 - b_1 = (a_2 - b_2) + (k - l)m$, so $m \mid [(a_1 - b_1) - (a_2 - b_2)]$.
- Multiplication: $a_1 b_1 = (a_2 + km)(b_2 + lm) = a_2 b_2 + a_2(lm) + b_2(km) + (kl)m^2 = a_2 b_2 + m(a_2 l + b_2 k + klm)$, so $m \mid (a_1 b_1 - a_2 b_2)$.

□

Corollary 2.1. *If $a \equiv b \pmod{m}$, then for any integer $k \geq 0$, $a^k \equiv b^k \pmod{m}$.*

Proof. By induction on k. Base: $k = 0$, $1 \equiv 1$. Assume for $k - 1$: $a^{k-1} \equiv b^{k-1} \pmod{m}$. Then $a^k = a \cdot a^{k-1} \equiv b \cdot b^{k-1} = b^k \pmod{m}$. □

Example 2.3.

Compute $6 + 9 \pmod{12}$: $6 + 9 = 15 \equiv 3 \pmod{12}$. Alternatively, $6 \equiv 18 \pmod{12}$, $9 \equiv -3 \pmod{12}$, so $18 + (-3) = 15 \equiv 3 \pmod{12}$.

For multiplication: $2 \cdot 3 = 6 \equiv 1 \pmod{5}$, since $2 \equiv 7 \pmod{5}$, $3 \equiv 8 \pmod{5}$, $7 \cdot 8 = 56 \equiv 1 \pmod{5}$ (as $55 = 11 \cdot 5$).

These properties allow us to perform arithmetic "modulo m" by reducing intermediates.

2.4 Cancellation and Division

Division in modular arithmetic is subtle and requires care, often involving multiplicative inverses. However, a cancellation law holds under certain conditions.

Proposition 2.2. *If $ac \equiv bc \pmod{m}$ and $d = \gcd(c, m)$, then $a \equiv b \pmod{m/d}$.*

Proof. Given $ac \equiv bc \pmod{m}$, we have $m \mid (ac - bc) = c(a - b)$. Let $d = \gcd(c, m)$. Write $c = dc'$, $m = dm'$ with $\gcd(c', m') = 1$. Then $dm' \mid dc'(a - b)$, so $m' \mid c'(a - b)$. Since $\gcd(c', m') = 1$, it follows that $m' \mid (a - b)$ (by Euclid's lemma), hence $a \equiv b \pmod{m' = m/d}$. □

Corollary 2.2. *If $\gcd(c, m) = 1$, then $ac \equiv bc \pmod{m}$ implies $a \equiv b \pmod{m}$.*

Example 2.4.

Suppose $4x \equiv 12 \pmod{10}$. Here $c = 4$, $m = 10$, and $\gcd(4, 10) = 2$. According to the cancellation proposition, we can divide both sides of the congruence by $d = 2$, provided we reduce the modulus accordingly. That is, the solutions satisfy:

$$x \equiv \frac{12}{2} \cdot (4/2)^{-1} \pmod{10/2} \quad \Rightarrow \quad x \equiv 6 \cdot 2^{-1} \pmod{5}$$

Since $2^{-1} \equiv 3 \pmod{5}$ (because $2 \cdot 3 = 6 \equiv 1 \pmod{5}$), we get:

$$x \equiv 6 \cdot 3 = 18 \equiv 3 \pmod{5}$$

So $x \equiv 3 \pmod{5}$ is the solution set.

Verification: choose $x = 3$, then $4x = 12 \equiv 2 \pmod{10}$. Wait! this doesn't match the original equation $4x \equiv 12 \pmod{10}$, because $12 \equiv 2 \pmod{10}$, so this suggests a mistake.
Actually, we need to reframe the initial congruence correctly: since $4x \equiv 12 \pmod{10}$ and $12 \equiv 2 \pmod{10}$, the congruence is really:

$$4x \equiv 2 \pmod{10}$$

Now apply the proposition: $\gcd(4, 10) = 2$ divides 2, so we can divide throughout by 2 to get:

$$2x \equiv 1 \pmod{5}$$

The inverse of 2 modulo 5 is 3 (since $2 \cdot 3 = 6 \equiv 1 \pmod{5}$), so:

$$x \equiv 3 \pmod{5}$$

Let's verify with $x = 3$:

$$4x = 12 \equiv 2 \pmod{10} \quad \text{correct.}$$

Now, consider a case where gcd = 1, such as $3x \equiv 6 \pmod{7}$. Since $\gcd(3, 7) = 1$, we can directly multiply both sides by the inverse of 3 modulo 7. Since $3^{-1} \equiv 5 \pmod{7}$ (because $3 \cdot 5 = 15 \equiv 1 \pmod{7}$), we get:

$$x \equiv 5 \cdot 6 = 30 \equiv 2 \pmod{7}$$

Verified: $3 \cdot 2 = 6 \equiv 6 \pmod{7}$ correct.

Note that if $\gcd(c, m) > 1$, cancellation may not hold modulo m, but only modulo m/d. This highlights why inverses exist only when coprime.

3 The Ring $\mathbb{Z}/m\mathbb{Z}$

Building on the concept of congruences from the previous section, we now formalize the algebraic structure formed by the residue classes modulo m. This structure, denoted $\mathbb{Z}/m\mathbb{Z}$, is a fundamental object in abstract algebra and number theory, serving as a prototype for finite rings and fields.

3.1 Construction and Ring Axioms

The set $\mathbb{Z}/m\mathbb{Z}$ consists of the m distinct residue classes modulo m:

$$\mathbb{Z}/m\mathbb{Z} = \{[0]_m, [1]_m, \ldots, [m-1]_m\},$$

where $[a]_m = \{b \in \mathbb{Z} : b \equiv a \pmod{m}\}$.

To make $\mathbb{Z}/m\mathbb{Z}$ into a ring, we define addition and multiplication on these classes:

$$[a]_m + [b]_m = [a+b]_m, \quad [a]_m \cdot [b]_m = [ab]_m.$$

These operations are **well-defined**, meaning they do not depend on the choice of representatives. If $[a'] = [a]$ and $[b'] = [b]$, then $a' \equiv a \pmod{m}$ and $b' \equiv b \pmod{m}$, so $a' + b' \equiv a + b \pmod{m}$ and $a'b' \equiv ab \pmod{m}$ by the arithmetic properties of congruences.

Theorem 3.1. *$(\mathbb{Z}/m\mathbb{Z}, +, \cdot)$ is a commutative ring with identity.*

Proof. The ring axioms are satisfied as follows:

- **Addition is associative and commutative**: Inherited from $\mathbb{Z}$, e.g., $([a] + [b]) + [c] = [a + b + c] = [a] + ([b] + [c])$.
- **Additive identity**: $[0]$, since $[a] + [0] = [a]$.
- **Additive inverses**: For $[a]$, the inverse is $[-a]$, since $[a] + [-a] = [0]$.
- **Multiplication is associative and commutative**: Similarly from $\mathbb{Z}$.
- **Multiplicative identity**: $[1]$, since $[a] \cdot [1] = [a]$.
- **Distributivity**: $[a] \cdot ([b] + [c]) = [a(b+c)] = [ab + ac] = [ab] + [ac] = [a] \cdot [b] + [a] \cdot [c]$.

Commutativity of both operations follows from $\mathbb{Z}$. □

Example 3.1. For $m = 5$, addition and multiplication tables in $\mathbb{Z}/5\mathbb{Z}$ are:

Addition Table:

$+$	0	1	2	3	4
0	0	1	2	3	4
1	1	2	3	4	0
2	2	3	4	0	1
3	3	4	0	1	2
4	4	0	1	2	3

Multiplication Table:

$\cdot$	0	1	2	3	4
0	0	0	0	0	0
1	0	1	2	3	4
2	0	2	4	1	3
3	0	3	1	4	2
4	0	4	3	2	1

$\mathbb{Z}/m\mathbb{Z}$ is the **quotient ring** of $\mathbb{Z}$ by the ideal $m\mathbb{Z}$.

3.2 Units and Zero Divisors

Not every non-zero element in $\mathbb{Z}/m\mathbb{Z}$ has a multiplicative inverse, leading to the distinction between units and zero divisors.

Definition 3.1. An element $[a] \in \mathbb{Z}/m\mathbb{Z}$ is a **unit** if there exists $[b]$ such that $[a]\cdot[b] = [1]$. The set of units is

$$(\mathbb{Z}/m\mathbb{Z})^* = \{[a] \in \mathbb{Z}/m\mathbb{Z} : \gcd(a, m) = 1\}.$$

Proposition 3.1. *$[a]$ is a unit if and only if $\gcd(a, m) = 1$.*

Proof. If $\gcd(a, m) = 1$, by Bézout's identity (later chapter), there exist integers u, v with $au + mv = 1$, so $[a][u] = [1]$. Conversely, if $[a][b] = [1]$, then $ab \equiv 1 \pmod{m}$, so $\gcd(a, m)$ divides 1. □

$(\mathbb{Z}/m\mathbb{Z})^*$ forms a group under multiplication.

Definition 3.2. A non-zero element $[a]$ is a **zero divisor** if there exists non-zero $[b]$ with $[a][b] = [0]$.

Proposition 3.2. *$[a] \neq [0]$ is a zero divisor if and only if $\gcd(a, m) > 1$.*

Proof. ($\Rightarrow$) Suppose $\gcd(a, m) = 1$; then $[a]$ is a unit, so if $[a][b] = [0]$, multiply by inverse: $[b] = [0]$, contradiction. ($\Leftarrow$) Let $d = \gcd(a, m) > 1$, $a = da'$, $m = dm'$ with $\gcd(a', m') = 1$. Then $[a][m'] = [da'm'] = [a'm] = [0]$, and $[m'] \neq [0]$ since $m' < m$. □

Example 3.2. In $\mathbb{Z}/6\mathbb{Z}$, units are $\{[1], [5]\}$ ($\gcd(1, 6) = 1$, $\gcd(5, 6) = 1$). Zero divisors: $[2]$ ($\gcd(2, 6) = 2 > 1$, $[2][3] = [6] = [0]$), $[3]$ ($\gcd(3, 6) = 3 > 1$), $[4]$ ($\gcd(4, 6) = 2 > 1$).

3.3 Field Structure for Prime Modulus

When $m = p$ is prime, $\mathbb{Z}/p\mathbb{Z}$ has additional structure.

Theorem 3.2. *If p is prime, then $\mathbb{Z}/p\mathbb{Z}$ is a field.*

Proof. Every non-zero $[a]$ has $\gcd(a, p) = 1$ (since p is prime and doesn't divide a), so all non-zero elements are units. There are no zero divisors (as that would require $\gcd(a, p) > 1$, impossible for $a \not\equiv 0$). Thus, it satisfies the field axioms: it's a commutative ring with identity where every non-zero element has an inverse. □

Fields like $\mathbb{Z}/p\mathbb{Z}$ (denoted $\mathbb{F}_p$) are crucial for finite field arithmetic in cryptography and coding theory.

Example 3.3. $\mathbb{Z}/5\mathbb{Z}$ is a field: every non-zero element has an inverse, e.g., $[2]^{-1} = [3]$ since $2 \cdot 3 = 6 \equiv 1 \pmod 5$.

3.4 Euler's Phi Function

To quantify the units, we introduce Euler's totient function.

Definition 3.3. Euler's **phi function** (totient function) is

$$\phi(m) = |(\mathbb{Z}/m\mathbb{Z})^*| = \#\{a : 1 \leq a \leq m, \gcd(a, m) = 1\}.$$

Example 3.4.

- $\phi(7) = 6$ (all 1 through 6 are coprime to 7).
- $\phi(24) = 8$ (1,5,7,11,13,17,19,23).

Theorem 3.3 (Euler's Product Formula)**.** *If $m = p_1^{k_1} \cdots p_r^{k_r}$, then*

$$\phi(m) = m \prod_{i=1}^{r} \left(1 - \frac{1}{p_i}\right) = \prod_{i=1}^{r} p_i^{k_i - 1}(p_i - 1).$$

Proof. For prime power p^k, $\phi(p^k) = p^k - p^{k-1}$ (subtract multiples of p). The general case follows from multiplicativity (below). □

Theorem 3.4 (Multiplicativity)**.** *If $\gcd(m, n) = 1$, then $\phi(mn) = \phi(m)\phi(n)$.*

Proof. By the Chinese Remainder Theorem (later chapter), there is a ring isomorphism $\mathbb{Z}/(mn)\mathbb{Z} \cong \mathbb{Z}/m\mathbb{Z} \times \mathbb{Z}/n\mathbb{Z}$, which restricts to a group isomorphism $(\mathbb{Z}/(mn)\mathbb{Z})^* \cong (\mathbb{Z}/m\mathbb{Z})^* \times (\mathbb{Z}/n\mathbb{Z})^*$. Thus, the orders multiply: $\phi(mn) = \phi(m)\phi(n)$. □

$\phi(m)$ counts integers up to m coprime to m and is key in Euler's theorem.

4 Extended Euclidean Algorithm & Inverses

The Euclidean algorithm and its extended form are essential tools in number theory for computing greatest common divisors (GCDs) and solving linear Diophantine equations. These algorithms are particularly useful in modular arithmetic for finding multiplicative inverses, which are crucial for division modulo m and applications in cryptography.

4.1 Euclidean Algorithm

The Euclidean algorithm efficiently computes the greatest common divisor $\gcd(a, b)$ of two integers a and b (assume $a \geq b > 0$ without loss of generality). It is based on the principle that $\gcd(a, b) = \gcd(b, a \mod b)$, repeatedly replacing the larger number with the remainder until a remainder of zero is reached.

Definition 4.1. The **greatest common divisor** $\gcd(a, b)$ is the largest positive integer d that divides both a and b.

Algorithm 1 Euclidean Algorithm

Require: Integers $a \geq b > 0$
Ensure: $\gcd(a, b)$
1: **while** $b \neq 0$ **do**
2: $q \leftarrow \lfloor a/b \rfloor$ ▷ quotient
3: $r \leftarrow a - qb$ ▷ remainder
4: $a \leftarrow b$
5: $b \leftarrow r$
6: **end while**
7: **return** a

Example 4.1. Compute $\gcd(252, 105)$:

$$\begin{aligned} 252 &= 2 \cdot 105 + 42, \\ 105 &= 2 \cdot 42 + 21, \\ 42 &= 2 \cdot 21 + 0. \end{aligned}$$

So $\gcd(252, 105) = 21$.

Theorem 4.1 (Correctness of Euclidean Algorithm)**.** *The algorithm computes* $\gcd(a, b)$ *in* $O(\log \min(a, b))$ *steps.*

Proof. subsubsection Let $r_0 = a$, $r_1 = b$, and define $r_{i+1} = r_{i-1} \mod r_i$. The sequence of remainders is strictly decreasing and non-negative: $r_{i+1} < r_i$. It terminates when $r_k = 0$, and $\gcd(a, b) = r_{k-1}$.

To show correctness: Any common divisor of a and b divides all remainders (since $r_{i+1} = r_{i-1} - q_i r_i$). Conversely, working backwards, any divisor of the final non-zero remainder divides a and b. Thus, $\gcd(a, b) = r_{k-1}$.

For complexity: Each step reduces the remainder by at least half in the worst case (Fibonacci numbers), leading to $O(\log \min(a, b))$. □

4.2 Extended Algorithm

The extended Euclidean algorithm augments the basic algorithm to find integers u and v satisfying Bézout's identity.

Algorithm 2 Extended Euclidean Algorithm

Require: Integers a, b
Ensure: $d = \gcd(a, b)$, and u, v such that $au + bv = d$

$u \leftarrow 1$, $v \leftarrow 0$
$u' \leftarrow 0$, $v' \leftarrow 1$
while $b \neq 0$ **do**
    $q \leftarrow \lfloor a/b \rfloor$
    $r \leftarrow a - q \cdot b$
    $u'' \leftarrow u - q \cdot u'$
    $v'' \leftarrow v - q \cdot v'$
    $a \leftarrow b$, $b \leftarrow r$
    $u \leftarrow u'$, $v \leftarrow v'$
    $u' \leftarrow u''$, $v' \leftarrow v''$
end while
return $d = a$, u, v

This tracks coefficients u and v through the remainders.

Theorem 4.2 (Bézout's Identity)**.** *For any integers* a, b*, there exist integers* u, v *such that* $au + bv = \gcd(a, b)$*.*

Proof. The proof is by induction on the steps of the Euclidean algorithm. Base case: If $b = 0$, then $\gcd(a, 0) = |a|$, and $a \cdot 1 + 0 \cdot 0 = a$ (adjust sign if needed).

Inductive step: Assume for smaller pairs. From $a = qb + r$, $\gcd(a, b) = \gcd(b, r)$. By induction, $bu' + rv' = \gcd(b, r)$. Substitute $r = a - qb$: $bu' + (a - qb)v' = av' + b(u' - qv') = \gcd(a, b)$. Thus $u = v'$, $v = u' - qv'$.

The extended algorithm implements this back-substitution efficiently. □

Example 4.2.

For $a = 252$, $b = 105$:

$$\begin{aligned} 252 &= 2 \cdot 105 + 42 \\ 105 &= 2 \cdot 42 + 21 \\ 42 &= 2 \cdot 21 + 0 \end{aligned}$$

So $\gcd(252, 105) = 21$. Now back-substituting to express 21 as a linear combination of 252 and 105:

$$\begin{aligned} 21 &= 105 - 2 \cdot 42 \\ &= 105 - 2 \cdot (252 - 2 \cdot 105) \\ &= 105 - 2 \cdot 252 + 4 \cdot 105 \\ &= (-2) \cdot 252 + 5 \cdot 105 \end{aligned}$$

Therefore, $252 \cdot (-2) + 105 \cdot 5 = 21 = \gcd(252, 105)$.

The algorithm runs in $O(\log \min(a, b))$ time, same as the basic version.

4.3 Computing Multiplicative Inverse

If $\gcd(a, m) = 1$, a has a multiplicative inverse modulo m: an integer u such that $au \equiv 1 \pmod{m}$. This u is unique modulo m.

From Bézout: $au + mv = 1$, so $au \equiv 1 \pmod{m}$.

Algorithm 3 Modular Inverse via Extended Euclidean

Require: a, m with $\gcd(a, m) = 1$.
Ensure: u such that $au \equiv 1 \pmod{m}$.
1: Use extended Euclidean on a, m to get u, v with $au + mv = 1$. **return** $u \mod m$ (adjust to positive if needed).

Example 4.3. Find the inverse of 5 modulo 12 (since $\gcd(5, 12) = 1$).
Apply Euclidean algorithm:

$$\begin{aligned} 12 &= 2 \cdot 5 + 2, \\ 5 &= 2 \cdot 2 + 1, \\ 2 &= 2 \cdot 1 + 0. \end{aligned}$$

Back-substitution:

$$\begin{aligned} &1 = 5 - 2 \cdot 2, \\ &2 = 12 - 2 \cdot 5, \\ &1 = 5 - 2 \cdot (12 - 2 \cdot 5) = 5 - 2 \cdot 12 + 4 \cdot 5 = 5 \cdot 5 - 2 \cdot 12. \end{aligned}$$

Thus, $5 \cdot 5 \equiv 1 \pmod{12}$, so the inverse is $5 \mod 12 = 5$. Verify: $5 \cdot 5 = 25 \equiv 1 \pmod{12}$.

Example 4.4. Inverse of 11 modulo 26:

$$\begin{aligned} 26 &= 2 \cdot 11 + 4, \\ 11 &= 2 \cdot 4 + 3, \\ 4 &= 1 \cdot 3 + 1, \\ 3 &= 3 \cdot 1 + 0. \end{aligned}$$

Back-substitution:

$$\begin{aligned} 1 &= 4 - 1 \cdot 3, \\ 3 &= 11 - 2 \cdot 4, \quad 1 && = 4 - 1 \cdot (11 - 2 \cdot 4) = 3 \cdot 4 - 1 \cdot 11, \\ 4 &= 26 - 2 \cdot 11, \quad 1 && = 3 \cdot (26 - 2 \cdot 11) - 1 \cdot 11 = 3 \cdot 26 - 7 \cdot 11. \end{aligned}$$

So $11 \cdot (-7) \equiv 1 \pmod{26}$, and $-7 \equiv 19 \pmod{26}$. Verify: $11 \cdot 19 = 209 = 8 \cdot 26 + 1 \equiv 1 \pmod{26}$.

If $\gcd(a, m) \neq 1$, no inverse exists.

This method is efficient, requiring $O(\log m)$ steps, vital for cryptographic computations.

5 Euler's Theorem and Applications

This section brings together the key results of Euler and Fermat, which underpin much of modern public-key cryptography, and demonstrates practical applications in fundamental cryptosystems.

5.1 Euler's Theorem

Theorem 5.1 (Euler's Theorem)**.** *Let $m \geq 1$ and let a be an integer with $\gcd(a, m) = 1$. Then*

$$a^{\varphi(m)} \equiv 1 \pmod{m},$$

where $\varphi(m)$ is Euler's totient function, the number of integers $1 \leq k \leq m$ that are coprime to m.

Proof. The set of invertible elements modulo m forms a multiplicative group of order $\varphi(m)$, namely $(\mathbb{Z}/m\mathbb{Z})^* = \{[b] : 1 \leq b \leq m, \gcd(b, m) = 1\}$. By Lagrange's theorem (from group theory), for any $[a]$ in this group,

$$[a]^{\varphi(m)} = [1],$$

which means $a^{\varphi(m)} \equiv 1 \pmod{m}$. □

5.2 Fermat's Little Theorem

As a special case of Euler's theorem, we have:

Theorem 5.2 (Fermat's Little Theorem)**.** *Let p be a prime. For any integer a not divisible by p,*

$$a^{p-1} \equiv 1 \pmod{p}.$$

Proof. For p prime, $\varphi(p) = p - 1$, so Euler's theorem applies directly: $a^{p-1} \equiv 1 \pmod{p}$ for $a \not\equiv 0 \pmod{p}$. □

5.3 Applications to Cryptography

Modular exponentiation theorems such as those of Euler and Fermat are the working engine inside multiple cryptosystems. We highlight two:

- **RSA Cryptosystem:**
  - *Key Generation:* Choose distinct large primes p, q; let $n = pq$, compute $\varphi(n) = (p-1)(q-1)$.
  - *Public Key:* (n, e) such that $1 < e < \varphi(n)$ and $\gcd(e, \varphi(n)) = 1$.
  - *Private Key:* d satisfying $ed \equiv 1 \pmod{\varphi(n)}$ (computed using the Extended Euclidean Algorithm).
  - *Encryption:* $c \equiv m^e \pmod{n}$.
  - *Decryption:* $m \equiv c^d \pmod{n}$.
  - *Correctness:* As $ed \equiv 1 \pmod{\varphi(n)}$ and $\gcd(m, n) = 1$, Euler's theorem gives $m^{\varphi(n)} \equiv 1 \pmod{n}$, so $(m^e)^d \equiv m \pmod{n}$.
- **Diffie–Hellman Key Exchange:**
  - Parties agree on a prime p and generator g of $(\mathbb{Z}/p\mathbb{Z})^*$.
  - Alice selects secret a and sends $A = g^a \pmod{p}$; Bob selects secret b and sends $B = g^b \pmod{p}$.
  - Alice computes $k = B^a \pmod{p}$; Bob computes $k = A^b \pmod{p}$.
  - By commutativity, $k = g^{ab} \pmod{p}$.
  - Security is based on the hardness of the discrete logarithm problem.

These theorems are thus not only of theoretical but also of immense practical importance, forming the mathematical basis of internet security and privacy protocols.

6 Multiplicative Order and Primitive Roots

6.1 Order of an Element

Definition 6.1. Let $m \geq 1$ be an integer, and a an integer coprime to m ($\gcd(a, m) = 1$). The **order** of a modulo m, denoted $\mathrm{ord}_m(a)$, is the smallest positive integer k such that

$$a^k \equiv 1 \pmod{m}.$$

Proposition 6.1. *For a with $\gcd(a, m) = 1$, $\mathrm{ord}_m(a)$ divides $\varphi(m)$.*

Proof. Since $a \in (\mathbb{Z}/m\mathbb{Z})^*$, it is an element of the finite multiplicative group of order $\varphi(m)$. By Lagrange's theorem in group theory, the order of an element divides the group's order, hence $\mathrm{ord}_m(a) \mid \varphi(m)$. □

6.2 Primitive Roots

Definition 6.2. A **primitive root modulo** m is an element g in $(\mathbb{Z}/m\mathbb{Z})^*$ whose order is equal to $\varphi(m)$. Equivalently, the powers of g generate the entire group $(\mathbb{Z}/m\mathbb{Z})^*$:

$$\{g, g^2, g^3, \ldots, g^{\varphi(m)} = 1\} = (\mathbb{Z}/m\mathbb{Z})^*.$$

Example 6.1. For $m = 7$, which is prime, $(\mathbb{Z}/7\mathbb{Z})^*$ is cyclic of order 6. The number 3 is a primitive root mod 7 because

$$3^1 = 3, \quad 3^2 = 2, \quad 3^3 = 6, \quad 3^4 = 4, \quad 3^5 = 5, \quad 3^6 = 1 \pmod{7}.$$

All non-zero residues appear in the powers of 3.

6.3 Existence Theorems

Theorem 6.1. *Primitive roots modulo m exist if and only if m is of the form:*

$$m = 1, 2, 4, p^k, \text{ or } 2p^k,$$

where p is an odd prime and $k \geq 1$.

Remark 6.1. For prime modulus p, the multiplicative group $(\mathbb{Z}/p\mathbb{Z})^*$ is cyclic, hence always has primitive roots.

Sketch. The proof uses structure theory of finite abelian groups and properties of unit groups modulo prime powers. The key is showing cyclicity of $(\mathbb{Z}/p\mathbb{Z})^*$ for prime p, then extending to prime powers and their doubles. Other moduli do not admit primitive roots. □

7 Chinese Remainder Theorem

Theorem 7.1 (Chinese Remainder Theorem (CRT))**.** *Let $m_1, m_2, \ldots, m_k$ be positive integers such that $\gcd(m_i, m_j) = 1$ for all $i \neq j$. For any integers $a_1, a_2, \ldots, a_k$, the system of simultaneous congruences*

$$x \equiv a_i \pmod{m_i}, \quad i = 1, 2, \ldots, k$$

has a unique solution x modulo $M = \prod_{i=1}^{k} m_i$.

7.1 Proof and Construction

Proof. Let $M = \prod_{i=1}^{k} m_i$ and for each i, define $M_i = M/m_i$.
Since $\gcd(M_i, m_i) = 1$, there exist integers y_i such that

$$M_i y_i \equiv 1 \pmod{m_i}.$$

Then, the solution is constructed as

$$x = \sum_{i=1}^{k} a_i M_i y_i \pmod{M}.$$

Indeed, for each j,

$$x \equiv a_j M_j y_j + \sum_{i \neq j} a_i M_i y_i \equiv a_j \cdot 1 + \sum_{i \neq j} a_i \cdot 0 = a_j \pmod{m_j},$$

since M_i is divisible by m_j for $i \neq j$.
Uniqueness follows because if x, x' are two solutions, then $x \equiv x' \pmod{m_i}$ for all i, so $m_i \mid (x - x')$ for all i, and thus $M \mid (x - x')$ by pairwise coprimality. □

7.2 Applications

- Simplifying computations modulo large composite numbers by splitting into computations modulo its prime-power factors.
- Cryptography: RSA uses CRT to speed up modular exponentiation by working mod p and q rather than $n = pq$.
- Computer algebra systems use CRT for reconstructing integer values from modular computations.
- Encoding multiple simultaneous modular constraints into a single congruence.

8 Fast Modular Exponentiation

In applications like RSA encryption or Diffie–Hellman key exchange, one must compute $g^A \pmod N$ where A and N can be very large (e.g., $A \approx 2^{2048}$, N with 2048 bits). The **naïve method** initializes $\texttt{res} = 1$ and multiplies by g exactly A times, reducing modulo N at each step:

$$\texttt{res} \leftarrow (\texttt{res} \cdot g) \bmod N, \quad \text{repeated } A \text{ times.}$$

This requires $A - 1$ multiplications, which is $O(A)$ time, impractical for large A, as it could take longer than the universe's age for $A = 2^{1000}$. The need for efficiency motivates **fast exponentiation** algorithms that reduce operations to $O(\log A)$, leveraging the exponent's binary representation.

8.1 Binary Exponentiation

The **binary exponentiation** (or **square-and-multiply**) algorithm computes $g^A \pmod N$ by expressing A in binary and using repeated squaring.

Algorithm 4 Fast Powering (Square-and-Multiply)

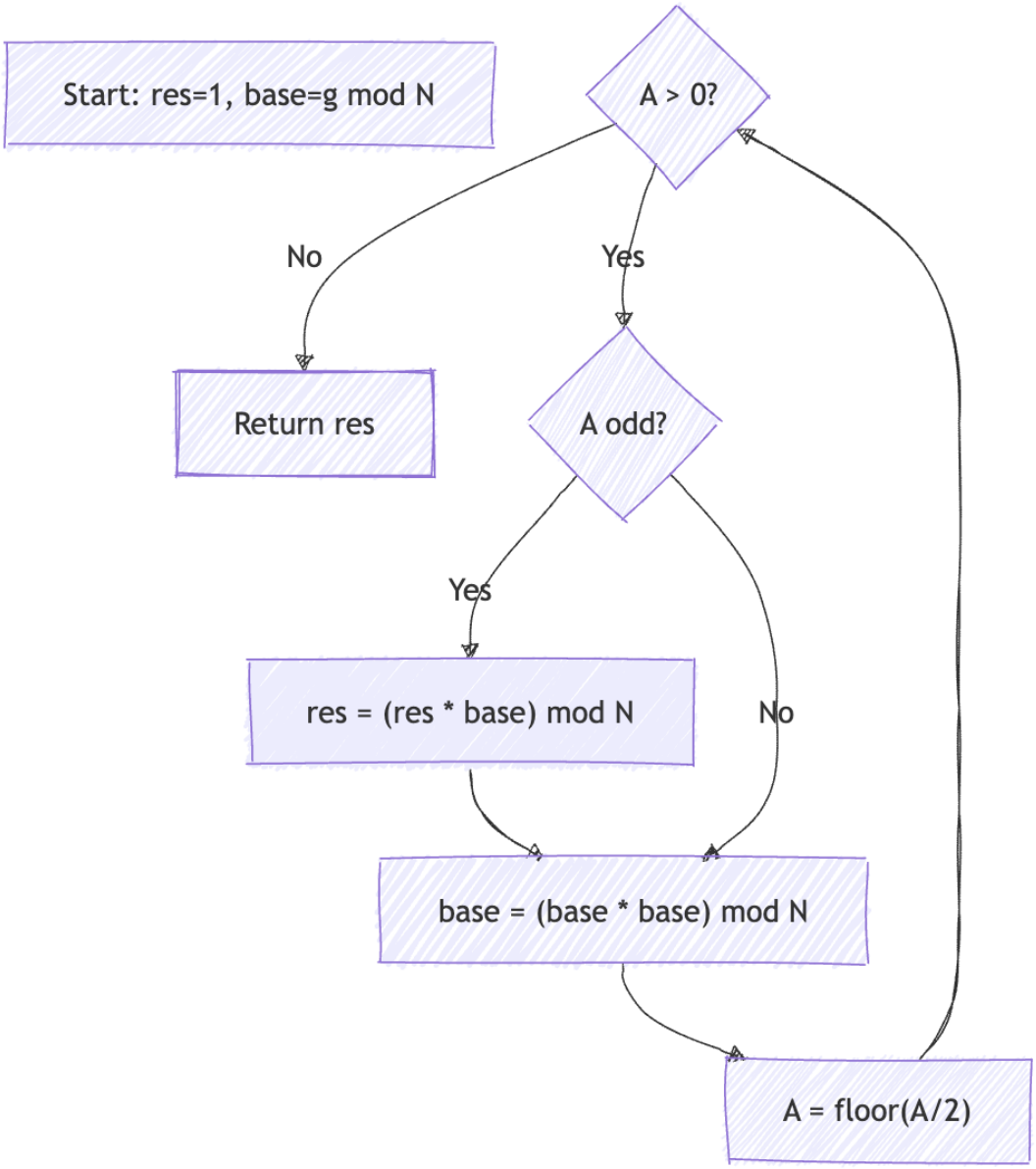
```
function MODEXP(g, A, N)
    res ← 1
    base ← g mod N
    while A > 0 do
        if A mod 2 = 1 then
            res ← (res · base) mod N
        end if
        base ← (base · base) mod N
        A ← ⌊A/2⌋
    end while
    return res
end function
```

This processes A's binary digits from least to most significant, squaring `base` each time and multiplying into `res` when the bit is 1.

Example 8.1.

We compute $3^{13} \bmod 10$ using binary exponentiation. Since $13 = 1101_2 = 8 + 4 + 1$:
Start with $\texttt{res} = 1$, $\texttt{base} = 3$, $A = 13$.
Since A is odd, update: $\texttt{res} = 1 \cdot 3 = 3$, square base: $3^2 = 9$, and halve exponent: $A = 6$.

Now $A = 6$ is even, so skip multiply. Square base: $9^2 = 81 \equiv 1 \pmod{10}$, update $A = 3$.
Then $A = 3$ is odd: $\texttt{res} = 3 \cdot 1 = 3$, base squared remains 1, $A = 1$.
Finally, $A = 1$ is odd again: $\texttt{res} = 3 \cdot 1 = 3$, and $A = 0$.
Thus, $3^{13} \equiv 3 \pmod{10}$. verified.



8.2 Proof of Correctness

Theorem 8.1. *The binary exponentiation algorithm correctly computes* $g^A \pmod{N}$.

Proof. We prove by induction on A.
Base Case: If $A = 0$, then res $= 1$, and $g^0 = 1$. If $A = 1$, one iteration occurs: A is odd, res becomes $1 \cdot g = g$, base is squared (unused), A becomes 0. correct.

Inductive Hypothesis: Assume correctness for all exponents less than A.
Inductive Step: Write $A = 2B + \delta$ where $\delta = A \bmod 2 \in \{0, 1\}$ and $B = \lfloor A/2 \rfloor < A$.

- If $\delta = 1$, we compute res $= 1 \cdot g$;
- base becomes g^2, A becomes B;
- recursively, we compute $(g^2)^B = g^{2B}$ and multiply by g if $\delta = 1$.

Therefore:

$$g^A = \begin{cases} g^{2B}, & \text{if } \delta = 0 \\ g \cdot g^{2B}, & \text{if } \delta = 1 \end{cases}$$

Each operation is done modulo N, so modular equivalence is preserved. □

8.3 Complexity Analysis

The loop runs $\lfloor \log_2 A \rfloor + 1$ times (number of bits in A).

- Each iteration includes 1 squaring (a multiplication).
- Additional multiplication if the bit is 1 (up to $\log_2 A + 1$, average is about $\frac{1}{2} \log_2 A$).

Time Complexity: $O(\log A)$ multiplications. Each multiplication is $O((\log N)^2)$ with schoolbook arithmetic, or faster with advanced methods (e.g., FFT).
Thus, total time is $O(\log A \cdot (\log N)^2)$.

Space Complexity: $O(\log N)$ to store intermediate values.
This allows computing $g^{2^{1000}} \pmod N$ in seconds on modern hardware.

8.4 Variants

8.4.1 Sliding Window Method

For large exponents, the exponent is scanned in k-bit windows:

- Precompute $g^0, g^1, \ldots, g^{2^k - 1}$.
- Scan exponent A in k-bit windows from most to least significant.
- Square k times per window; multiply by precomputed value.

This reduces the number of multiplications to approximately $(1 + \frac{1}{k}) \log_2 A$.
Optimal window size is roughly $k \approx \log \log A$.
This is efficient for fixed-base scenarios like RSA public exponentiation.

8.4.2 Montgomery Multiplication

Montgomery multiplication transforms numbers into a special form:

$$x \mapsto xR \pmod{N}, \quad \text{where } R \text{ is typically } 2^k > N$$

In this form, multiplications avoid costly divisions by N, using only shifts and additions.

Montgomery form is ideal for repeated modular multiplications, as used in modular exponentiation.

9 Advanced Topics

9.1 Addition Chains

Addition chains provide efficient methods to compute large powers with minimal multiplications by building sequences where each element is a sum of two previous elements.

Definition 9.1. An **addition chain** for an integer n is a sequence:

$$a_0 = 1, a_1, a_2, \ldots, a_k = n$$

such that each a_i for $i > 0$ can be written as $a_i = a_j + a_m$ for some $j, m < i$.

Addition chains aim to minimize k, the number of multiplications for exponentiation.

9.2 Quadratic Residues and Reciprocity

Definition 9.2. For an odd prime p, an integer a is a **quadratic residue modulo** p if there exists x such that

$$x^2 \equiv a \pmod{p}.$$

Otherwise, a is a **quadratic nonresidue modulo** p.

Definition 9.3. The **Legendre symbol** $\left(\frac{a}{p}\right)$ is defined as:

$$\left(\frac{a}{p}\right) = \begin{cases} 0 & \text{if } p \mid a, \\ 1 & \text{if } a \text{ is a quadratic residue mod } p, \\ -1 & \text{if } a \text{ is a quadratic nonresidue mod } p. \end{cases}$$

Theorem 9.1 (Quadratic Reciprocity)**.** *For distinct odd primes p and q,*

$$\left(\frac{p}{q}\right)\left(\frac{q}{p}\right) = (-1)^{\frac{p-1}{2}\cdot\frac{q-1}{2}}.$$

9.3 Number-Theoretic Transform

The Number-Theoretic Transform (NTT) is a finite field analog of the Fast Fourier Transform (FFT), used for polynomial multiplication modulo a prime.

- Requires a prime modulus p where $p = kn + 1$ for n a power of 2.
- Uses a primitive n-th root of unity modulo p.
- Facilitates convolution via $\text{NTT}^{-1}(\text{NTT}(a) \times \text{NTT}(b))$.

9.4 Elliptic Curve Exponentiation

Elliptic curve cryptography relies on exponentiation in groups formed by points on an elliptic curve over finite fields.

- The group law on curves gives point addition.
- Scalar multiplication $kP = P + P + \cdots + P$ (k times) analogous to exponentiation.
- Fast algorithms similar to binary exponentiation apply for scalar multiplication.
- Provides stronger security per key length compared to RSA.